POLITÉCNICO DE LEIRIA | ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Undergraduate in Games and Multimedia | 2025/2026

Advanced Programming Topics

REPORT

Project1-Xenon Clone Game Engine

Students

Dinis Stasyuk | 2230673

Class Diagrams

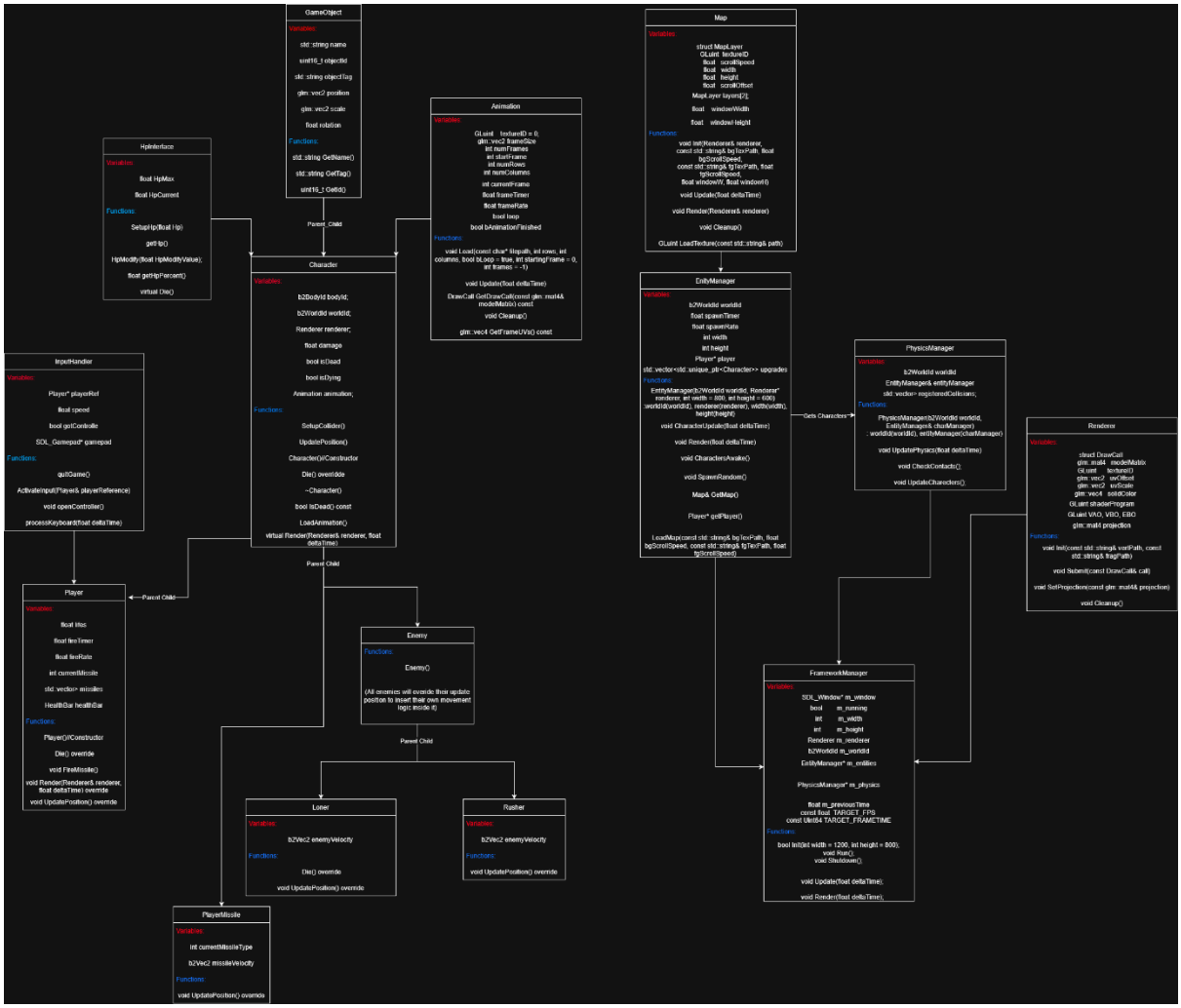

Figure 1-Class diagram of the Xenon clone with Game Engine, available in the files with full resolution

FrameWorkManager

FrameWorkManager is responsible for uniting all the major scripts and managing all game content to produce the game, it runs in the main the following way:

```
#pragma once
#include <ctime>
#include <iostream>
#include "FrameWorkManager.h"

int main()
{
    FrameworkManager framework;
    //Setup and initialazation provide with window size
    if (!framework.Init(1200, 800)) return -1;

    //Loop
    framework.Run();
    //Clear and end
    framework.Shutdown();

    return 0;
}
```

First the manager initializes a window with size the provided, also gets a controller, it will run all code to verify if none of the external libraries cause any issues, creates Renderer, setups Box2d, EnityManager, PhysicsManager and sets run to true. (Will go in further detail later on for the EntityManager and PhysicsManager.)

Second it executes Run() which executes every frame while running it manages FPS capping them, deltaTime providing it to Update and Render and checks if player did not die or for an sdlevent Quit.

Update runs the respective updates from PhysicsManager and EnityManager which to the following:

PhysicsManager -PlayerInput → physics step → position sync box to sprite→ checks contacts

EntityManager- Checks if needs to Spawn → out-of-bounds KILL → add missiles from player → despawns

Render it clears, than it runs Renderer inside the EntityManager so every “entity” gets rendered and gets m_renderer.Present() necessary for this sdl implementation.

With the rendering, entities and physics separated the order of execution for the gameEngine should be the following first a “Awake” start the player and map, player being necessary for the PhysicsManager, run loop with fps, UpdatePhysics, UpdateEntities and than Updating Graphics.

EntityManager

Note that Character is the pawn class which in this current project most objects inherit from.

EntityManager is responsible for holding and spawning every entity, which include enemy, player, missiles, map, rendering it deleting it on death and providing it to other classes that might need it such as the PhysicsManager which requires all characters.

First we run the AwakePlayer() which creates player and stores it than we create the map, we need the player to exist before the physics manager starts so it is the first object created.

We have than the CharacterUpdate, update first updates map, checks with the timer if it is time to spawn a random Enemy, kills any Character outside of the map, player won't ever have this problem since they have a position clamp, it adds any player missiles to the character list so they can also be Updated and Rendered, it than removes from storage any characters that are dead unless it's the player and at last Render inside the EnityManager is triggered.

Render triggers render first inside map than inside all other characters, in the current implementation each object runs their "own" rendering, (they will just utilize the rendering class for rendering but some classes due it in their own way so for safety it triggered this way).

SpawnRandom gets a random location within both width and height and spawns enemies, with a random roll, providing the enemies with the wordl id and sldRenderer as well as position accordingly.

EntityManager has also a reference to input handler but only activates it this inpuHandler will be provided to the PhysicsManager to update physics, entities and rendering at separate instances, ensuring consistency.

PhysicsManager

PhysicsManager requires both EntityManager and worldId on construct, it has the following functions:

UpdateCharacter- its simple it gets the characters from EntityManager and updates off their positions the UpdatePosition function is inside all members of Character class and it updates physics inside the characters.

Check Contacts -First it gets all sensor events, clears the vector registeredCollisions, since event collision registers each collision twice with different order and we want to consider each collision only once.

Next, we check every collision event and verify if the pair of objects as been already registered for collision if yes move on to the next collision pair else. Perform a cast to character on both of these objects since we only character objects in this project to check for collisions. Next we check if any of these object is dead or dying since we won't need consider their collisions (tough collisions should be already deactivated if the character is dead or dying, we just confirm it).

Next we get the tags from both objects if tags are different, we proceed and if we verify that one of the objects has the tag "Enemy" and the other has the tag "Player" both objects damage each other, with HpModify from HealthInterface, in which case means that Player and PlayerMissiles, damage Enemies and vise-versa. Lastly we register the collision in registeredCollisions independently of events.

The update was already mentioned previously runs this way in scripts - entityManager.input.processKeyboard(deltaTime); → b2World_Step(worldId, 1.0f / 60.0f, 4); → UpdateCharecters();→ CheckContacts();.

```
void PhysicsManager::CheckContacts()
{
    //Get all  SensorEvents tried with GetContactEvents but did not work
    b2SensorEvents events = b2World_GetSensorEvents(worldId);

    //Empties collisions each frame
    registeredCollisions.clear();

    //Check for all start of collisions
    for (int i = 0; i < events.beginCount; ++i)
    {
        //Provides both of the bodies that collide
        b2SensorBeginTouchEvent event = events.beginEvents[i];

        // Sensor events give you sensorShapeId and visitorShapeId
        void* sensorData = b2Body_GetUserData(b2Shape_GetBody(event.sensorShapeId));
        void* visitorData = b2Body_GetUserData(b2Shape_GetBody(event.visitorShapeId));


        //First check if both objects are Character since in this part only collision between characters matter
        Character* objectA = dynamic_cast<Character*>((Character*)sensorData);
        Character* objectB = dynamic_cast<Character*>((Character*)visitorData);

        //Registers collisions
        registeredCollisions.push_back({ objectA, objectB });


        //Verify first if both objects are Characters since both enemies and players are and we only need to check for them
        if (!objectA || !objectB) continue;

        //verify if collision isn't verified a second time in the same frame just reverse order
        bool alreadyRegistered = false;
        for (auto& [a, b] : registeredCollisions) {
            if ((a == objectA && b == objectB) || (a == objectB && b == objectA)) {
                alreadyRegistered = true;
                break; //Can't to continue here it will just break this for but not the next one
            }
        }
        if (alreadyRegistered) continue;

        //Confirm if the object isn't dead or dying
        if (objectA->isDying || objectA->isDead) continue;
        if (objectB->isDying || objectB->isDead) continue;

        std::string tagA = objectA->getTag();
        std::string tagB = objectB->getTag();

        //If tags are == there is no interaction since objects with the same tag don't interact with each other
        if (tagA == tagB) continue;

        //We verify if the enemy/enemyMissile enters in contact with player/playerMissile
        if ((tagA == "Player" || tagB == "Player") && (tagA == "Enemy" || tagB == "Enemy"))
        {
            //Yes both objects damage each other on contact the values are programmed in a way so missiles always get destroyed on contact, while player destroys anything they touch
            objectA->HpModify(-objectB->damage);
            objectB->HpModify(-objectA->damage);
            std::cout << "Object With name " << objectA->getName() << " with tag " << objectA->getTag() << " has " << objectA->getHp() << " Hp left" << std::endl;
            std::cout << "Object With name " << objectB->getName() << " with tag " << objectB->getTag() << " has " << objectB->getHp() << " Hp left" << std::endl;
        }
    }
}
```

InputHandler

Input Handler is run in the physics as shown previously it requires a reference to the player to run. Compared to the previous scripts it is very simple the only thing that is called outside are the void ActivateInput(Player* playerReference) { playerRef = playerReference; }; and processKeyboard(float deltaTime), process keyboard runs a OpenController function which seeks the first controller, there is no separate logic for controller and keyboard though only the arrows work for controller movement.

There is also the special action which attempts fire the missile on the player and a quit on Escape.

```
void InputHandler::processKeyboard(float deltaTime)
{
    if (!gotController)
    {
        openController();
        gotController = true;
    }
    if (!playerRef) return;

    b2Vec2 playerVelocity = { 0.0f, 0.0f };
    const bool* keyState = SDL_GetKeyboardState(NULL);

    //Only Dpad movement for this part of the project
    if (keyState[SDL_SCANCODE_W] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_UP))playerVelocity.y = -speed;
    if (keyState[SDL_SCANCODE_S] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_DOWN))playerVelocity.y = speed;
    if (keyState[SDL_SCANCODE_A] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_LEFT))playerVelocity.x = -speed;
    if (keyState[SDL_SCANCODE_D] || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_DPAD_RIGHT))playerVelocity.x = speed;

    //Adding player movement trough box 2D since we will have to account phisics into our movement calcs
    b2Body_SetLinearVelocity(playerRef->bodyId, playerVelocity);

    //Keys for special action
    if (keyState[SDL_SCANCODE_SPACE] || keyState[SDL_SCANCODE_LSHIFT] || keyState[SDL_SCANCODE_RSHIFT]
        || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_LEFT_SHOULDER) || SDL_GetGamepadButton(gamepad, SDL_GAMEPAD_BUTTON_RIGHT_SHOULDER)
        || SDL_GetGamepadAxis(gamepad, SDL_GAMEPAD_AXIS_LEFT_TRIGGER) || SDL_GetGamepadAxis(gamepad, SDL_GAMEPAD_AXIS_RIGHT_TRIGGER))
    {
        playerRef->FireMissile();
    }

    //Quit game
    if (keyState[SDL_SCANCODE_ESCAPE]) { std::cout << "Quit Game" << std::endl; quitGame(); }
}
```

Animation

Imports sprites with file path, number of rows and columns which it uses in its calculation to get the amount of frames and the size of each sprite, there can be set manually the number of frames in the animation if it loops. The texture it loads if it a magenta pixel it gets replaced with transparency. Also has public bool used to check if death animation is finished. In this implementation with sdl required to get renderer.

Renderer

Renderer in this project is used with sdl there is nothing in particular special about it most logic is done outside it mostly submits draw calls, and if there is no texture places a red rectangle instead.

Map

Map has constantly 2 tiles scrolling can set scrolling speed of each one manually receives the speed, renderer and both background textures paths. It isn't particularly efficient nor good and if the background has a magenta pixel does not remove it.

HpInterface

The HpInterface is very simple it is provided to all Characters it has 3 getters a Die which is meant to be overrided by the objects that own it and HpModify which modifies hp both for healing and taking damage, it can't be above the MaxHp and if the player Hp reaches =< 0 it triggers the Die event.

GameObject and Character

The main class we have in this project is the GameObject class this class contains the variables, name, objectId, objectTag, position, scale, rotation and is the ancestor for most of the objects of this project and serves as the parent to the main class Character.

Character class is the main class for all objects with, collisions, animation and health and it sets them up with the help of the HpInterface, Animation class and box2d.

Character adds variables and functions for collision, rendering, damage, animation and for dying/dead.

It is obligatory to provide the character with a renderer and a world Id the rest has defaults, also all children of Character need to provide their own spriteSheet for the animation to load.

For collisions a box collider2D sensor is made that follows the sprite and position of the character with the help of UpdatePosition.

Rendering inside Character updates animation, submits draws inside the renderer, and prevents the player from being dead while the death animation is played.

The Die from healthInterface is override so that the characters trigger a death animation and while dying they don't have any colliders as well as stops their movement.

Player

Player inherits from character and adds the fire missile, and adds a clamp to their position based on maps limits which it receives on construct, the player speed is set by the input handler outside of player which runs in physics and player infiltrates the updates of the Renderer to run a timer for missile fireRate and to erase any dead missiles.

PlayerMissile

Just has the update position and a set velocity to go Up, most of the logic involving it is outside of it be it in the CheckContacts or the spawning inside the player. Important that this has the tag "Player" and 1 hp so dies on colliding with any setup object.

Enemy

Enemies inherit from character, the enemy class initially had more functionality, but gradually through the process of this work it lost many of the things that distinguished it from other characters since its functionality was added to the character class, and at the moment it is used by the only the Loner and Rusher which are both initialized by the SpawnRandom inside EntityManager.

Loners spawns from the left moving to the right while Rusher spawns from above moving

Conclusion

The only thing that wasn't done was the separation between the gameEngine and the xenon clone theoretically, it could have been done by making a new class for the Player and PlayerMissile and giving the FrameWorkManager another a Player Child, but my laptop drivers corrupted and ran out of time. I also forgot to add the second background image it works just need to add it.

References

The Cherno. (2019, September 15). *How to Build a 2D Renderer | Game Engine series* [Video]. YouTube. https://www.youtube.com/watch?v=ULO8JhM9FsQ, used many other videos

Lazy Foo' Productions - Beginning Game Programming v2.0. (n.d.). https://lazyfoo.net/tutorials/SDL/

Technologies, U. (n.d.-b). *Unity - Manual: Execution Order of event functions*. https://docs.unity3d.com/560/Documentation/Manual/ExecutionOrder.html

Technologies, U. (n.d.-c). *Unity - Manual: Fundamental Unity types*. https://docs.unity3d.com/6000.3/Documentation/Manual/fundamental-unity-types.html

Simple Direct Media Layer. (n.d.). SDL_GamepadButton. SDL Wiki.

https://wiki.libsdl.org/SDL3/SDL_GamepadButton

As well as the materials provided in class, and I manually looked through some UE5 directories for some inspiration.